

SYSTEM REPORT & DEPLOYMENT GUIDE

Miss Belle Blog

A comprehensive system report and deployment guide for the Miss Belle spoken word artist blog. Covers architecture, performance optimizations, technology stack, and step-by-step production deployment instructions for VPS and cloud platforms.

PREPARED FOR

Moh Santity Ankimbim — missbelle.art

JUNE 2026

Table of Contents

1. System Overview	3
1.1 Key Features	3
2. Technology Stack	4
2.1 Core Framework	4
2.2 Styling and UI	4
2.3 Full Dependency Table	5
3. Architecture and Performance Optimizations	6
3.1 Code Splitting with next/dynamic	6
3.2 Targeted CSS Transitions	6
3.3 Animation Cleanup	7
3.4 Theme Toggle Timer Fix	7
4. Project Structure	7
5. Prerequisites and Environment Setup	8
5.1 Build Machine Requirements	8
5.2 Deployment Server Requirements	8

6. Local Development	9
6.1 Setup Steps	9
6.2 Development Scripts	10
7. Production Build	10
7.1 Build Command	10
7.2 Build Output Structure	11
8. Deployment Methods	11
8.1 Method A: VPS with PM2 (Recommended for Control) . . .	11
8.2 Method B: Docker Container	12
8.3 Method C: Vercel (Easiest, Zero-Config)	12
8.4 Method D: Caddy Reverse Proxy	13
9. Environment Variables and Configuration	13
10. Post-Deployment Verification	14
10.1 Verification Checklist	14
10.2 Performance Testing	15
11. Troubleshooting Common Issues	15

1. System Overview

The Miss Belle blog is a single-page marketing website built for Moh Santity Ankimbim, a spoken word artist and poet performing under the stage name Miss Belle. The site serves as her professional online presence, showcasing performances, testimonials, gallery images, and providing a booking form for event organizers. It is designed with a strong emphasis on visual storytelling, dark-mode aesthetics with gold accents, and smooth animations that reflect the artistic nature of spoken word poetry.

The website is a client-side rendered React application built with Next.js 16 (App Router) and styled using Tailwind CSS 4 with the shadcn/ui component library. It features bilingual support for English and French through a custom internationalization provider, light and dark theme toggling via next-themes, and performance-critical optimizations including code-split lazy loading for all below-the-fold sections. The application is configured for standalone output, meaning it produces a self-contained production build that includes its own Node.js server, eliminating the need for a traditional Node.js installation on the deployment target.

The primary domain for this project is missbelle.art. The site includes structured data (JSON-LD schema markup for a Person entity), Open Graph and Twitter card metadata for social media sharing, a progressive web app manifest, and SEO-friendly robots.txt configuration. Two API routes handle form submissions: a booking inquiry endpoint with Zod validation and honeypot spam protection, and a newsletter subscription endpoint. Both routes currently log submissions to the console and include TODO markers for integration with email services such as Resend, Formspree, or Buttondown.

1.1 Key Features

The blog implements a rich set of interactive and presentational features designed to engage visitors and communicate the artist's brand effectively. Below is a summary of the primary feature areas, each implemented as a self-contained React component loaded dynamically to minimize the initial JavaScript bundle size and improve page load performance.

Feature	Description	Component
Hero Section	Full-viewport landing with typewriter animation, waveform visualizer, and scroll progress indicator	hero.tsx, scroll-progress.tsx
Navigation	Sticky top navigation bar with responsive mobile menu and language toggle	navigation.tsx, language-toggle.tsx
Statistics	Animated counters displaying performance metrics and career highlights	stats-counter.tsx

Feature	Description	Component
About	Biography section with image, text, and animated entry	about.tsx
Performances	Event listing with date, venue, and booking integration	performances.tsx
Testimonials	Rotating testimonial carousel with Framer Motion transitions	testimonials.tsx
Gallery	Photo grid with lightbox viewer using AnimatePresence	gallery.tsx
Booking Form	Multi-field booking inquiry with Zod validation and honeypot	booking.tsx
Newsletter	Email subscription with consent checkbox and validation	newsletter.tsx
Theme Toggle	Dark/light mode switch with smooth CSS transitions	theme-toggle.tsx

2. Technology Stack

The project leverages a modern JavaScript/TypeScript stack centered around the Next.js framework. All dependencies and their versions are managed through the package.json manifest, with lock files generated by the Bun package manager. The technology choices prioritize developer experience, performance, and a rich visual design system suitable for an artist's portfolio website. Below is a comprehensive breakdown of every major technology category and its role within the application.

2.1 Core Framework

Next.js 16.1.1 serves as the application framework, using the App Router architecture introduced in Next.js 13 and stabilized through subsequent releases. The App Router provides file-system-based routing, server components by default, and nested layouts. The project is configured with `output: 'standalone'`, which produces an optimized production build that includes a minimal Node.js server capable of running the application without installing the full `node_modules` tree on the target server. TypeScript 5 provides static type checking across all source files, with build-time type errors suppressed via `ignoreBuildErrors: true` in the Next.js configuration for faster iteration during development.

2.2 Styling and UI

Tailwind CSS 4 is used for utility-first styling, configured with custom color tokens defined as HSL CSS variables (such as `--background`, `--foreground`, `--card`, `--primary`, and so on). This approach enables seamless theme switching between light and dark modes by toggling a CSS class on the HTML root element. The shadcn/ui component library provides a comprehensive set of pre-built, accessible UI primitives (buttons, dialogs, forms, tooltips, etc.) built on top of Radix UI. Framer Motion 12 handles all animations and transitions throughout the site, including page-level scroll progress, section entry animations, testimonial carousel transitions, and gallery lightbox effects.

2.3 Full Dependency Table

Catagory	Package	Vers ion	Purpose
Frame work	next	16.1. 1	React meta-framework with App Router
Runtim e	react / react-dom	19.0. 0	UI rendering library
Styling	tailwindcss	4.x	Utility-first CSS framework
UI Library	radix-ui (multiple)	vario us	Accessible headless UI primitives
Animat ions	framer-motion	12.23 .2	Declarative animation library
Themi ng	next-themes	0.4.6	Dark/light mode management
Fonts	next/font (Google Fonts)	built- in	Inter, Playfair Display, Great Vibes
Validat ion	zod	4.0.2	Schema validation for API routes
Databa se	prisma / @prisma/client	6.11. 1	ORM for SQLite database access
Icons	lucide-react	0.525 .0	SVG icon library
Toasts	sonner	2.0.6	Toast notification system
Packag e Mgr	bun	lock file	Fast JavaScript package manager

3. Architecture and Performance Optimizations

Performance was a primary concern during the development of this blog. The original implementation loaded all section components synchronously, resulting in a large initial JavaScript bundle and slow time-to-interactive metrics. Three major optimizations were applied to address these issues: code splitting via dynamic imports, elimination of CSS paint thrashing caused by wildcard transition selectors, and removal of expensive Framer Motion layout animations. Each optimization is detailed below with the specific code changes and the performance impact they deliver.

3.1 Code Splitting with `next/dynamic`

The primary performance bottleneck was the synchronous import of eight section components (StatsCounter, About, Performances, Testimonials, Gallery, Booking, Newsletter, and Footer) at the top level of the main page. This forced the browser to download, parse, and execute the JavaScript for every section before the page became interactive, even though only the Hero and Navigation sections are visible above the fold. The solution was to wrap each below-fold section in a `next/dynamic()` call with `ssr: false` and a placeholder loading element that reserves the correct vertical space to prevent layout shift.

This approach means the initial HTML sent by the server contains only the critical above-the-fold content (Hero, Navigation, ScrollProgress, BackToTop). All other sections are loaded as separate JavaScript chunks that are fetched asynchronously by the browser after the initial page render. The loading placeholders use CSS height values matching each section's approximate rendered height, which prevents cumulative layout shift (CLS) as sections load progressively into the page. The build output confirms multiple chunk files are generated, with the main bundle significantly smaller than the original monolithic build.

3.2 Targeted CSS Transitions

The original stylesheet applied a wildcard CSS transition rule to every DOM element using `*, *::before, *::after { transition-property: background-color, color, border-color, ... }`. This rule forced the browser's transition machinery to activate on every single element in the document tree, including deeply nested divs, spans, and pseudo-elements that never actually change their visual properties. On a page with hundreds of DOM nodes (common with complex Tailwind utility output), this caused significant paint thrashing during theme toggling and page load, as the browser had to compute transitions for elements that were not visually changing.

The fix replaced the wildcard selector with a set of targeted selectors scoped to elements that actually participate in theme transitions. The new rule uses a `html.theme-transition` class toggled by the theme switch button, and applies transitions only to `body`, `main`, `header`, `footer`, `section`, and elements with background-related CSS classes. This reduced the number of elements participating in

transitions from potentially thousands to approximately two dozen, eliminating the paint thrashing entirely.

3.3 Animation Cleanup

Framer Motion's `AnimatePresence` component was configured with `mode='popLayout'` and child elements used the `layout` prop. The `layout` prop instructs Framer Motion to automatically animate an element's position and size whenever the DOM layout changes, which triggers continuous layout recalculations during list reordering or element mounting. Combined with `popLayout'` mode (which exits and enters elements simultaneously), this caused expensive reflow operations during testimonial carousel transitions and gallery lightbox animations.

The fix was straightforward: set `initial={false}` on `AnimatePresence` to prevent the initial mount animation (which was causing a visible flash on page load), remove all `layout` props from animated children, and reduce transition durations from 0.5s to 0.25s for gallery items and 0.4s for testimonials. These changes eliminated the layout thrashing while maintaining smooth, visually appealing transitions for section changes.

3.4 Theme Toggle Timer Fix

A subtle but impactful bug existed in the theme toggle component. The original code used a cleanup function inside the `onClick` handler to clear a `setTimeout`, following the pattern `return () => clearTimeout(timer)`. However, React event handlers do not treat return values as cleanup functions; the returned function was silently discarded. This meant the timer that removes the `theme-transition` CSS class was never properly cleared if the user clicked the toggle rapidly, causing the class to persist and break subsequent theme switches. The fix replaced the inline timer with a `useRef` that stores the timer ID, allowing the new click handler to clear any pending timer before setting a new one, ensuring consistent transition behavior regardless of click frequency.

4. Project Structure

The project follows the standard Next.js App Router directory convention, with all source code organized under the `src/` directory. The structure separates concerns into clear directories: page components in `src/app/`, reusable section components in `src/components/sections/`, UI primitives from shadcn/ui in `src/components/ui/`, custom animations in `src/components/animations/`, internationalization files in `src/lib/i18n/`, and API route handlers in `src/app/api/`. The Prisma schema and database files reside at the project root. Below is the complete directory tree with annotations for the most important files.

```
src/app/page.tsx - Main page component with dynamic imports
src/app/layout.tsx - Root layout with fonts, ThemeProvider, metadata, JSON-LD
src/app/globals.css - Global styles, CSS variables, theme transitions
```

```

src/app/api/booking/route.ts - Booking form POST handler (Zod validated)
src/app/api/newsletter/route.ts - Newsletter subscription handler
src/components/sections/ - All page section components (hero, about, gallery, etc.)
src/components/ui/ - shadcn/ui primitives (button, dialog, form, etc.)
src/components/animations/ - Typewriter, fade-in, waveform effects
src/lib/i18n/ - English and French translation JSON files, I18nProvider
prisma/schema.prisma - Database schema (User, Post models, SQLite)
next.config.ts - Next.js config (standalone output, no TS errors)
tailwind.config.ts - Tailwind with HSL CSS variable colors, class dark mode
Caddyfile - Caddy reverse proxy config for port 3000
public/ - Static assets (favicon, OG image, manifest, robots.txt)

```

5. Prerequisites and Environment Setup

Before building or deploying the Miss Belle blog, your development machine or server must meet the following requirements. The standalone build mode significantly reduces the runtime dependencies on the deployment server, as it bundles its own minimal Node.js runtime. However, the build process itself requires a full development environment with Node.js (or Bun), npm (or Bun), and all project dependencies installed.

5.1 Build Machine Requirements

Requirement	Minimum Version	Recommended	Notes
Node.js	18.17.0	20.x or 22.x LTS	Required for Next.js 16; use nvm or fnm for version management
Package Manager	npm 9.x	Bun 1.x	Project uses bun.lock; Bun is significantly faster
Git	2.30+	Latest	For cloning the repository
Disk Space	500 MB	1 GB	Includes node_modules, .next build output, and standalone server
OS	Linux x64	Ubuntu 22.04+ / Debian 12+	Standalone output targets Linux; macOS works for development

5.2 Deployment Server Requirements

Because the project uses `output: 'standalone'`, the deployment server does not need Node.js or npm installed. The standalone build produces a self-contained server at `.next/standalone/server.js` that includes only the minimal dependencies required to run. However, you still need a JavaScript runtime on the deployment machine to execute this server file. The options are listed below, along with a summary of what gets deployed to the server.

Option	Runtime Needed	Pros	Cons
Node.js	Node.js 18+	Full compatibility, standard approach	Must install and maintain Node.js
Bun	Bun 1.x	Faster startup, used in dev	Less common on hosting platforms
Docker	None (bundled)	Reproducible, portable	Larger image size (~150-200 MB)
Vercel	None (managed)	Zero-config, auto-scaling	Vendor lock-in, less control

6. Local Development

Setting up the project for local development is straightforward. Clone the repository, install dependencies, and start the development server. The development server supports hot module replacement (HMR), meaning changes to any source file are reflected instantly in the browser without a full page reload. This section covers the complete setup process step by step, from cloning the repository to verifying the development server is running.

6.1 Setup Steps

Open a terminal in your project directory (after downloading and extracting the files) and run the following commands in sequence. If you are using Visual Studio Code, you can open the integrated terminal with `Ctrl+`` (backtick) and run these commands directly.

```
# Step 1: Install dependencies
npm install
# Or if you have Bun installed (recommended, faster):
bun install

# Step 2: (Optional) Set up the database
npx prisma generate
npx prisma db push

# Step 3: Start the development server
npm run dev
```

```
# Or with Bun:
bun run dev

# Step 4: Open in browser
# Navigate to http://localhost:3000
```

6.2 Development Scripts

Script	Command	Description
dev	next dev -p 3000	Start development server with HMR on port 3000
build	next build + copy static	Production build with standalone output and asset copying
start	NODE_ENV=production bun .next/standalone/server.js	Start production server from standalone build
lint	eslint .	Run ESLint on all source files
db:push	prisma db push	Push Prisma schema to SQLite database
db:generate	prisma generate	Generate Prisma client from schema

7. Production Build

The production build process compiles the Next.js application into an optimized, deployment-ready artifact. The build command in package.json performs three operations in sequence: first, it runs **next build** which compiles all pages and components, tree-shakes unused code, and generates the standalone server; second, it copies the **.next/static** directory into the standalone output; and third, it copies the **public/** directory into the standalone output. These copy steps are necessary because the standalone output does not automatically include static assets and public files.

7.1 Build Command

```
# Run the production build
npm run build

# The build output will be in:
# .next/standalone/ - Self-contained server + minimal node_modules
# .next/standalone/.next/static/ - JS/CSS bundles
# .next/standalone/public/ - Static assets (favicon, OG images, manifest)
```

7.2 Build Output Structure

After a successful build, the `.next/standalone/` directory contains everything needed to run the application in production. This directory is fully self-contained and can be copied to any server with a compatible runtime. The structure includes the standalone server entry point, a minimal set of `node_modules` containing only the production dependencies, the compiled page and API route handlers, and the static assets that were copied during the build step. The total size of the standalone directory is typically between 80 and 150 megabytes, depending on the number of dependencies.

```
.next/standalone/
|-- server.js # Entry point: run with `node server.js` or `bun server.js`
|-- .next/
| |-- server/ # Compiled server-side pages and API routes
| |-- static/ # Hashed JS/CSS chunks (copied from .next/static/)
|-- node_modules/ # Minimal production dependencies only
|-- public/ # Static assets (copied from project root public/)
|-- package.json # Minimal package.json for the standalone server
```

8. Deployment Methods

The Miss Belle blog can be deployed using several strategies, ranging from a simple VPS with a manual process script to fully managed platforms that handle scaling and SSL automatically. This chapter covers the four recommended deployment methods in detail, each with complete step-by-step instructions. The choice of method depends on your budget, technical expertise, and expected traffic volume. For most personal artist websites, a basic VPS deployment or Vercel is the most appropriate choice.

8.1 Method A: VPS with PM2 (Recommended for Control)

Deploying to a Virtual Private Server gives you full control over the runtime environment, domain configuration, and process management. This method uses PM2, a production process manager for Node.js, to keep the application running and automatically restart it if it crashes. The steps assume you have a Linux server (Ubuntu 22.04 or Debian 12) with SSH access and a registered domain name pointing to your server's IP address.

```
# On your LOCAL machine - build and upload
npm run build
cd .next/standalone
tar -czf missbelle-deploy.tar.gz .
scp missbelle-deploy.tar.gz user@your-server:/home/user/

# On your SERVER - extract and run
mkdir -p /var/www/missbelle
cd /var/www/missbelle
tar -xzf /home/user/missbelle-deploy.tar.gz

# Install PM2 globally and start the app
```

```
npm install -g pm2
pm2 start server.js --name missbelle
pm2 save
pm2 startup # generates command to auto-start on boot
```

8.2 Method B: Docker Container

Docker provides a fully portable and reproducible deployment environment. The container image includes the standalone server, all required files, and a minimal Linux base, making it easy to deploy to any platform that supports Docker (AWS ECS, Google Cloud Run, DigitalOcean App Platform, or any VPS with Docker installed). Create a Dockerfile in the project root with the following content, build the image locally, and push it to a container registry or deploy it directly.

```
# Dockerfile (place in project root)
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json bun.lock ./
RUN npm install
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000
COPY --from=builder /app/.next/standalone ./
EXPOSE 3000
CMD ["node", "server.js"]
```

8.3 Method C: Vercel (Easiest, Zero-Config)

Vercel is the recommended hosting platform for Next.js applications and provides the simplest deployment experience. Simply connect your Git repository (GitHub, GitLab, or Bitbucket) to Vercel, and every push to the main branch triggers an automatic build and deployment. Vercel handles SSL certificates, CDN distribution, serverless function execution, and automatic scaling without any configuration. The free tier is sufficient for a personal artist website. Note that Vercel ignores the output: 'standalone' setting and uses its own optimized deployment pipeline, so no changes to the codebase are needed.

```
# Deploy to Vercel (requires Vercel CLI)
npm i -g vercel
vercel login
vercel --prod

# Or connect via the Vercel web dashboard:
# 1. Go to vercel.com and sign in with GitHub
# 2. Click 'New Project' and import your repository
# 3. Vercel auto-detects Next.js and deploys
```

```
# 4. Add custom domain: missbelle.art in Project Settings
```

8.4 Method D: Caddy Reverse Proxy

If you already have a VPS running the standalone server on port 3000, you can use Caddy as a reverse proxy to handle HTTPS termination, automatic SSL certificate provisioning via Let's Encrypt, and static file serving. The project includes a Caddyfile in the repository root that is configured to proxy requests to localhost:3000. To use it, install Caddy on your server, copy the Caddyfile to the appropriate location, and update the domain name. Caddy will automatically obtain and renew SSL certificates.

```
# Install Caddy on Ubuntu/Debian
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg
--dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo
tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install caddy

# Copy Caddyfile and restart
sudo cp Caddyfile /etc/caddy/Caddyfile
# Edit the file to replace ':81' with 'missbelle.art'
sudo systemctl restart caddy
sudo systemctl enable caddy
```

9. Environment Variables and Configuration

The application currently requires only one environment variable for full functionality: the database connection URL used by Prisma. All other configuration is handled through hardcoded values in the source code (such as the domain name in metadata and the theme default). The table below lists the current and planned environment variables, including those that will be needed when integrating email services for the booking and newsletter API routes. When deploying, create a `.env` file in the project root (or the standalone directory) with the required variables.

Variable	Re qui red	Default	Description
DATABASE_URL	Yes	file:./db/custom.db	SQLite connection string for Prisma ORM
PORT	No	3000	Port for the standalone server to listen on
HOSTNAME	No	0.0.0.0	Hostname to bind (use 0.0.0.0 for all interfaces)

Variable	Required	Default	Description
RESEND_API_KEY	Planned	-	API key for Resend email service (booking notifications)
BUTTONDOWN_API_KEY	Planned	-	API key for Buttondown newsletter service
CONTACT_EMAIL	Planned	-	Destination email for booking form submissions

10. Post-Deployment Verification

After deploying the application to your server, perform the following verification checks to ensure everything is functioning correctly. These checks cover the core functionality of the site, including page rendering, interactivity, theme switching, form submissions, and SEO metadata. Run through each check systematically and address any issues before considering the deployment complete.

10.1 Verification Checklist

Check	How to Verify	Expected Result
Page loads	Open missbelle.art in browser	Hero section renders with typewriter animation
Dark/Light mode	Click the sun/moon icon in navigation	Theme switches smoothly without flash
Language toggle	Click EN/FR toggle in navigation	All visible text switches language
Scroll animations	Scroll down the page slowly	Sections animate in as they enter viewport
Gallery lightbox	Click any gallery image	Lightbox opens with smooth transition
Booking form	Fill out and submit the booking form	Success toast notification appears
Newsletter	Enter email and subscribe	Success message appears

Check	How to Verify	Expected Result
Mobile responsive	Open in mobile browser or DevTools	All sections stack vertically, menu collapses
SEO meta	View page source or use browser DevTools	Title, description, OG tags, JSON-LD present
SSL/HTTPS	Check browser address bar	Green lock icon, https:// prefix

10.2 Performance Testing

Use Google Lighthouse (built into Chrome DevTools) or web.dev/measure to run a performance audit on the deployed site. With the code-splitting optimizations in place, you should see a Performance score above 85, with First Contentful Paint under 1.5 seconds and Largest Contentful Paint under 2.5 seconds on a 4G mobile connection. If scores are lower, check that the standalone build was created correctly with the static assets copied into the standalone directory. Also verify that the server is running in production mode (`NODE_ENV=production`), as development mode is significantly slower and should never be used in production.

11. Troubleshooting Common Issues

This chapter addresses the most common issues you may encounter during development, building, and deployment. Each issue includes a description of the problem, the likely root cause, and the specific steps to resolve it. If you encounter an issue not covered here, check the Next.js documentation at nextjs.org/docs and the project's worklog at worklog.md for additional context about known issues and their resolutions.

Issue	Likely Cause	Solution
Build fails with module not found	Missing dependencies	Run <code>npm install</code> or <code>bun install</code> to reinstall all packages
Blank page after deployment	Static assets not copied	Ensure the build script copies <code>.next/static</code> and <code>public/</code> into <code>.next/standalone/</code>
Theme flash on page load	Missing <code>disableTransitionOnChange</code>	Verify <code>ThemeProvider</code> has <code>disableTransitionOnChange={true}</code> in <code>layout.tsx</code>

Issue	Likely Cause	Solution
Font not loading (squares/garbled)	Google Fonts blocked or slow	Check network tab; fonts use display:swap so fallback fonts show first
Port 3000 already in use	Another process on port	Kill the process: <code>lsof -i :3000</code> then <code>kill -9 PID</code> , or change port
API returns 500 error	Missing environment variable	Check DATABASE_URL is set in .env file at project root
CLS (layout shift) on load	Loading placeholders wrong height	Adjust the height values in the <code>dynamic()</code> loading callbacks in <code>page.tsx</code>
CSS transitions not smooth	Wildcard selector still present	Ensure <code>globals.css</code> uses targeted selectors, not <code>*</code> , <code>*::before</code> , <code>*::after</code>